

Estructuras de Datos, 2026-2

Proyecto del curso

Avance: viernes 30 de octubre de 2026

Entrega final: lunes 16 de noviembre de 2026, 23:59

Sustentaciones: 18 y 25 de noviembre

De qué se trata

El proyecto es individual y pide resolver un problema real escogiendo las estructuras de datos, implementando una de ellas desde cero y midiendo si lo que cuesta en la práctica coincide con lo que dice el análisis. Vale el 30 % del curso: el avance 10 y la entrega final con sustentación 20.

Hay que escoger uno de los cuatro escenarios de la sección siguiente. Cada escenario nombra dos estructuras que resuelven el mismo problema con costos distintos; el proyecto las implementa, las compara midiendo y explica la diferencia con lo visto en clase.

Los cuatro escenarios

- A. Inventario de una tienda.** Cada artículo tiene código, nombre y cantidad. Operaciones: agregar, eliminar por código, consultar por código, listar todo y reponer existencias. *Se comparan:* una lista enlazada escrita desde cero contra **vector** de la biblioteca. La pregunta que se mide: qué cuesta eliminar e insertar en medio cuando el inventario crece, y qué cuesta recorrerlo.
- B. Frecuencias de palabras de un texto.** Se lee un texto grande y se cuenta cuántas veces aparece cada palabra distinta. Operaciones: insertar o incrementar, consultar una palabra y listar las k más frecuentes. *Se comparan:* una tabla hash con encadenamiento escrita desde cero contra **map** de la biblioteca. La pregunta que se mide: cómo cambian las colisiones y el tiempo de consulta al variar el tamaño de la tabla y el factor de carga.
- C. Turnos de una clínica.** Los pacientes llegan, esperan y se atienden; los urgentes pasan antes que los demás. Operaciones: registrar llegada, atender al siguiente, consultar cuántos esperan y el tiempo promedio de espera. *Se comparan:* una cola circular escrita desde cero, que atiende por orden de llegada, contra una cola de prioridad. La pregunta que se mide: cómo cambia el tiempo de espera de los urgentes y qué cuesta cada operación en los dos casos.
- D. Índice de títulos.** Un catálogo de títulos de libros o de canciones que se consulta por nombre. Operaciones: insertar, buscar exacto, eliminar y listar en orden alfabético. *Se comparan:* un árbol binario de búsqueda escrito desde cero contra **set** de la biblioteca. La pregunta que se mide: cómo cambia la altura del árbol según el orden en que se insertan los títulos, y qué le pasa a la búsqueda cuando el árbol degenera.

La estructura escrita desde cero es la primera de cada pareja y no puede delegar en la biblioteca estándar: se implementa con arreglos, punteros o nodos, según corresponda.

Qué se entrega

El trabajo vive en un repositorio de GitHub. El repositorio base está en <https://github.com/EjerciciosClasesCardestr-2026-2-proyecto>: hágale *fork* y trabaje sobre su copia. El archivo `AUTOR.md` del fork lleva su nombre completo y su código; sin eso el repositorio no se puede asociar a nadie.

Se califica el último commit anterior a la hora de cierre de cada entrega. La historia de commits cuenta: un repositorio con un solo commit el día del cierre no muestra avance.

Avance — viernes 30 de octubre

1. **Informe corto** (de 3 a 5 páginas, en `informe/`): el escenario escogido descrito en sus propias palabras; qué guarda la estructura y qué operaciones necesita; por qué escogió esa estructura y qué costo espera de cada operación, citando el texto guía; y qué piensa medir.
2. **La estructura escrita desde cero**, con sus operaciones principales funcionando (en `codigo/`).
3. **Casos de prueba**: al menos tres funcionales y uno límite (estructura vacía, un solo elemento, estructura llena, clave repetida), con la salida esperada y la obtenida.
4. **Bitácora** (`bitacora.md`): una entrada fechada por sesión de trabajo, con qué hizo y qué quedó pendiente.

Entrega final — lunes 16 de noviembre, 23:59

1. **Las dos estructuras** resolviendo el mismo problema, con la misma interfaz de operaciones.
2. **Análisis formal** del costo en tiempo y en espacio de cada operación, con la estructura de cuatro partes: teorema, estrategia, desarrollo y conclusión con testigos c y k . Los invariantes se escriben como fórmulas.
3. **Experimentos**: mida las dos estructuras con entradas de tamaños crecientes, al menos cinco tamaños, con la misma máquina y el mismo compilador. Reporte los tiempos en una tabla y en una gráfica, y diga si lo medido coincide con lo que predijo el análisis; cuando no coincida, explique por qué.
4. **Informe final** (en `informe/`, LaTeX o Markdown): el del avance corregido, más el análisis, los experimentos y las conclusiones. Los diagramas de las estructuras se dibujan, no se describen.
5. **Sustentación** de 10 minutos: se explica el modelado, se muestra el código corriendo y se responden preguntas sobre las decisiones y los costos.

Cómo se califica

Las dos entregas se califican sobre 5,0. El avance vale el 10 % del curso y la final con la sustentación, el 20 %.

Entrega	Criterio	Puntos
Avance	Modelado: el escenario, las operaciones y la estructura escogida, con la justificación y los costos esperados	30
	Implementación desde cero: las operaciones principales funcionan y respetan las restricciones de código	40
	Casos de prueba, incluido el límite, con salida esperada y obtenida	20
	Bitácora con entradas fechadas	10
Final	Las dos estructuras completas y con la misma interfaz	30
	Análisis formal de complejidad en tiempo y espacio	20
	Experimentos, con tabla, gráfica y contraste contra el análisis	20
	Informe: claridad, diagramas, citas al texto guía	15
	Sustentación	15

Quien no sustente recibe, como máximo, la quinta parte de la nota de la entrega final. Un proyecto cuyo código no compila no recibe los puntos de implementación ni los de experimentos.

Restricciones de código

Las mismas del curso, y se califican:

- C o C++, compilado con `gcc -Wall -Wextra` o `g++ -Wall -Wextra`, sin advertencias.
- Solo biblioteca estándar. La estructura escrita desde cero no usa contenedores de la STL por dentro.
- Sin `break`, sin `continue`, sin `goto`, y un solo `return` al final de cada función.
- Toda reserva con `malloc` o `new` va con su liberación, y el retorno de `malloc` se verifica.
- El código compila con el `Makefile` del repositorio: `make` produce los ejecutables y `make pruebas` corre los casos.

Integridad

El proyecto es individual. Discutir ideas con un compañero está bien; compartir código no. Las entregas se comparan entre sí con herramientas de similitud y se revisan con un detector de patrones de código generado. El código escrito íntegramente por herramientas de inteligencia artificial se considera plagio y puede derivar en un proceso ante la Dirección de Programa. Cite en el informe cualquier fuente que haya consultado.